JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class AbstractList<E>

java.lang.Object
    java.util.AbstractCollection<E>
        java.util.AbstractList<E>

All Implemented Interfaces:

`Iterable<E>, Collection<E>, List<E>`

Direct Known Subclasses:

`AbstractSequentialList, ArrayList, Vector`

```
public abstract class AbstractList<E>
extends AbstractCollection<E>
implements List<E>
```

This class provides a skeletal implementation of the `List` interface to minimize the effort required to implement this interface backed by a "random access" data store (such as an array). For sequential access data (such as a linked list), `AbstractSequentialList` should be used in preference to this class.

To implement an unmodifiable list, the programmer needs only to extend this class and provide implementations for the `get(int)` and `size()` methods.

To implement a modifiable list, the programmer must additionally override the `set(int, E)` method (which otherwise throws an `UnsupportedOperationException`). If the list is variable-size the programmer must additionally override the `add(int, E)` and `remove(int)` methods.

The programmer should generally provide a void (no argument) and collection constructor, as per the recommendation in the `Collection` interface specification.

Unlike the other abstract collection implementations, the programmer does *not* have to provide an iterator implementation; the iterator and list iterator are

implemented by this class, on top of the "random access" methods: `get(int)`, `set(int, E)`, `add(int, E)` and `remove(int)`.

The documentation for each non-abstract method in this class describes its implementation in detail. Each of these methods may be overridden if the collection being implemented admits a more efficient implementation.

This class is a member of the Java Collections Framework.

Since:

1.2

Field Summary

Fields

Modifier and Type	Field	Description
`protected int`	**`modCount`**	The number of times this list has been *structurally modified*.

Constructor Summary

Constructors

Modifier	Constructor	Description
`protected`	**`AbstractList()`**	Sole constructor.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
`boolean`	**`add`**`(E e)`	Appends the specified element to the end of this list (optional operation).
`void`	**`add`**`(int index, E element)`	Inserts the specified element at the specified position in this list (optional operation).

boolean	**addAll**(int index, **Collection**<? extends **E**> c)	Inserts all of the elements in the specified collection into this list at the specified position (optional operation).
void	**clear**()	Removes all of the elements from this list (optional operation).
boolean	**equals**(**Object** o)	Compares the specified object with this list for equality.
abstract **E**	**get**(int index)	Returns the element at the specified position in this list.
int	**hashCode**()	Returns the hash code value for this list.
int	**indexOf**(**Object** o)	Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.
Iterator<**E**>	**iterator**()	Returns an iterator over the elements in this list in proper sequence.
int	**lastIndexOf**(**Object** o)	Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element.
ListIterator<**E**>	**listIterator**()	Returns a list iterator over the elements in this list (in proper sequence).
ListIterator<**E**>	**listIterator**(int index)	Returns a list iterator over the elements in this list (in proper

		sequence), starting at the specified position in the list.
E	**remove**(int index)	Removes the element at the specified position in this list (optional operation).
protected void	**removeRange**(int fromIndex, int toIndex)	Removes from this list all of the elements whose index is between `fromIndex`, inclusive, and `toIndex`, exclusive.
E	**set**(int index, **E** element)	Replaces the element at the specified position in this list with the specified element (optional operation).
List<**E**>	**subList**(int fromIndex, int toIndex)	Returns a view of the portion of this list between the specified `fromIndex`, inclusive, and `toIndex`, exclusive.

Methods inherited from class java.util.AbstractCollection

addAll, contains, containsAll, isEmpty, remove, removeAll, retainAll, size, toArray, toArray, toString

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.List

addAll, contains, containsAll, isEmpty, remove, removeAll, replaceAll, retainAll, size, sort, spliterator, toArray, toArray

Methods inherited from interface java.util.Collection

parallelStream, removeIf, stream

Methods inherited from interface java.lang.Iterable

`forEach`

Field Detail

modCount

```
protected transient int modCount
```

The number of times this list has been *structurally modified*. Structural modifications are those that change the size of the list, or otherwise perturb it in such a fashion that iterations in progress may yield incorrect results.

This field is used by the iterator and list iterator implementation returned by the `iterator` and `listIterator` methods. If the value of this field changes unexpectedly, the iterator (or list iterator) will throw a `ConcurrentModificationException` in response to the `next`, `remove`, `previous`, `set` or `add` operations. This provides *fail-fast* behavior, rather than non-deterministic behavior in the face of concurrent modification during iteration.

Use of this field by subclasses is optional. If a subclass wishes to provide fail-fast iterators (and list iterators), then it merely has to increment this field in its `add(int, E)` and `remove(int)` methods (and any other methods that it overrides that result in structural modifications to the list). A single call to `add(int, E)` or `remove(int)` must add no more than one to this field, or the iterators (and list iterators) will throw bogus `ConcurrentModificationExceptions`. If an implementation does not wish to provide fail-fast iterators, this field may be ignored.

Constructor Detail

AbstractList

```
protected AbstractList()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Detail

add

`public boolean add(E e)`

Appends the specified element to the end of this list (optional operation).

Lists that support this operation may place limitations on what elements may be added to this list. In particular, some lists will refuse to add null elements, and others will impose restrictions on the type of elements that may be added. List classes should clearly specify in their documentation any restrictions on what elements may be added.

This implementation calls `add(size(), e)`.

Note that this implementation throws an `UnsupportedOperationException` unless `add(int, E)` is overridden.

Specified by:

`add` in interface `Collection<E>`

Specified by:

`add` in interface `List<E>`

Overrides:

`add` in class `AbstractCollection<E>`

Parameters:

`e` - element to be appended to this list

Returns:

`true` (as specified by `Collection.add(E)`)

Throws:

`UnsupportedOperationException` - if the add operation is not supported by this list

`ClassCastException` - if the class of the specified element prevents it from being added to this list

`NullPointerException` - if the specified element is null and this list does not permit null elements

`IllegalArgumentException` - if some property of this element prevents it from being added to this list

get

`public abstract E get(int index)`

Returns the element at the specified position in this list.

Specified by:

`get` in interface `List<E>`

Parameters:

`index` - index of the element to return

Returns:

the element at the specified position in this list

Throws:

`IndexOutOfBoundsException` - if the index is out of range (index < 0 || index >= size())

set

```
public E set(int index,
             E element)
```

Replaces the element at the specified position in this list with the specified element (optional operation).

This implementation always throws an `UnsupportedOperationException`.

Specified by:

`set` in interface `List<E>`

Parameters:

`index` - index of the element to replace

`element` - element to be stored at the specified position

Returns:

the element previously at the specified position

Throws:

`UnsupportedOperationException` - if the set operation is not supported by this list

`ClassCastException` - if the class of the specified element prevents it from being added to this list

`NullPointerException` - if the specified element is null and this list does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this list

`IndexOutOfBoundsException` - if the index is out of range (index < 0 || index >= size())

add

```
public void add(int index,
                E element)
```

Inserts the specified element at the specified position in this list (optional operation). Shifts the element currently at that position (if any) and any subsequent elements to the right (adds one to their indices).

This implementation always throws an `UnsupportedOperationException`.

Specified by:

`add` in interface `List<E>`

Parameters:

`index` - index at which the specified element is to be inserted

`element` - element to be inserted

Throws:

`UnsupportedOperationException` - if the add operation is not supported by this list

`ClassCastException` - if the class of the specified element prevents it from being added to this list

`NullPointerException` - if the specified element is null and this list does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this list

`IndexOutOfBoundsException` - if the index is out of range (index < 0 || index > size())

remove

```
public E remove(int index)
```

Removes the element at the specified position in this list (optional operation). Shifts any subsequent elements to the left (subtracts one from their indices). Returns the element that was removed from the list.

This implementation always throws an `UnsupportedOperationException`.

Specified by:

`remove` in interface `List<E>`

Parameters:

`index` - the index of the element to be removed

Returns:

the element previously at the specified position

Throws:

`UnsupportedOperationException` - if the remove operation is not supported by this list

`IndexOutOfBoundsException` - if the index is out of range (index < 0 || index >= size())

indexOf

```
public int indexOf(Object o)
```

Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element. More formally, returns the lowest index `i` such that `(o==null ? get(i)==null : o.equals(get(i)))`, or -1 if there is no such index.

This implementation first gets a list iterator (with `listIterator()`). Then, it iterates over the list until the specified element is found or the end of the list is reached.

Specified by:

`indexOf` in interface `List<E>`

Parameters:

`o` - element to search for

Returns:

the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element

Throws:

`ClassCastException` - if the type of the specified element is incompatible with this list (`optional`)

`NullPointerException` - if the specified element is null and this list does not permit null elements (`optional`)

lastIndexOf

```
public int lastIndexOf(Object o)
```

Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element. More formally, returns the highest index `i` such that `(o==null ? get(i)==null : o.equals(get(i)))`, or -1 if there is no such index.

This implementation first gets a list iterator that points to the end of the list (with `listIterator(size())`). Then, it iterates backwards over the list until the specified element is found, or the beginning of the list is reached.

Specified by:

`lastIndexOf` in interface `List<E>`

Parameters:

`o - element to search for`

Returns:

`the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element`

Throws:

`ClassCastException - if the type of the specified element is incompatible with this list (optional)`

`NullPointerException - if the specified element is null and this list does not permit null elements (optional)`

clear

```
public void clear()
```

Removes all of the elements from this list (optional operation). The list will be empty after this call returns.

This implementation calls `removeRange(0, size())`.

Note that this implementation throws an `UnsupportedOperationException` unless `remove(int index)` or `removeRange(int fromIndex, int toIndex)` is overridden.

Specified by:

`clear` in interface `Collection<E>`

Specified by:

`clear` in interface `List<E>`

Overrides:

`clear` in class `AbstractCollection<E>`

Throws:

`UnsupportedOperationException - if the clear operation is not supported by this list`

addAll

```
public boolean addAll(int index,
                      Collection<? extends E> c)
```

Inserts all of the elements in the specified collection into this list at the specified position (optional operation). Shifts the element currently at that position (if any) and any subsequent elements to the right (increases their indices). The new elements will appear in this list in the order that they are returned by the specified collection's iterator. The behavior of this operation is undefined if the specified collection is modified while the operation is in progress. (Note that this will occur if the specified collection is this list, and it's nonempty.)

This implementation gets an iterator over the specified collection and iterates over it, inserting the elements obtained from the iterator into this list at the appropriate position, one at a time, using `add(int, E)`. Many implementations will override this method for efficiency.

Note that this implementation throws an `UnsupportedOperationException` unless `add(int, E)` is overridden.

Specified by:

`addAll` in interface `List<E>`

Parameters:

`index` - index at which to insert the first element from the specified collection

`c` - collection containing elements to be added to this list

Returns:

`true` if this list changed as a result of the call

Throws:

`UnsupportedOperationException` - if the addAll operation is not supported by this list

`ClassCastException` - if the class of an element of the specified collection prevents it from being added to this list

`NullPointerException` - if the specified collection contains one or more null elements and this list does not permit null elements, or if the specified collection is null

`IllegalArgumentException` - if some property of an element of the specified collection prevents it from being added to this list

`IndexOutOfBoundsException` - if the index is out of range (index < 0 || index > size())

iterator

`public Iterator<E> iterator()`

Returns an iterator over the elements in this list in proper sequence.

This implementation returns a straightforward implementation of the iterator interface, relying on the backing list's `size()`, `get(int)`, and `remove(int)` methods.

Note that the iterator returned by this method will throw an `UnsupportedOperationException` in response to its `remove` method unless the list's `remove(int)` method is overridden.

This implementation can be made to throw runtime exceptions in the face of concurrent modification, as described in the specification for the (protected) `modCount` field.

Specified by:

`iterator` in interface `Iterable<E>`

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in interface `List<E>`

Specified by:

`iterator` in class `AbstractCollection<E>`

Returns:

an iterator over the elements in this list in proper sequence

listIterator

`public ListIterator<E> listIterator()`

Returns a list iterator over the elements in this list (in proper sequence).

This implementation returns `listIterator(0)`.

Specified by:

`listIterator` in interface `List<E>`

Returns:

a list iterator over the elements in this list (in proper sequence)

See Also:

`listIterator(int)`

listIterator

```
public ListIterator<E> listIterator(int index)
```

Returns a list iterator over the elements in this list (in proper sequence), starting at the specified position in the list. The specified index indicates the first element that would be returned by an initial call to `next`. An initial call to `previous` would return the element with the specified index minus one.

This implementation returns a straightforward implementation of the `ListIterator` interface that extends the implementation of the `Iterator` interface returned by the `iterator()` method. The `ListIterator` implementation relies on the backing list's `get(int)`, `set(int, E)`, `add(int, E)` and `remove(int)` methods.

Note that the list iterator returned by this implementation will throw an `UnsupportedOperationException` in response to its `remove`, `set` and `add` methods unless the list's `remove(int)`, `set(int, E)`, and `add(int, E)` methods are overridden.

This implementation can be made to throw runtime exceptions in the face of concurrent modification, as described in the specification for the (protected) `modCount` field.

Specified by:

`listIterator` in interface `List<E>`

Parameters:

`index - index of the first element to be returned from the list iterator (by a call to next)`

Returns:

`a list iterator over the elements in this list (in proper sequence), starting at the specified position in the list`

Throws:

`IndexOutOfBoundsException - if the index is out of range (index < 0 || index > size())`

subList

```
public List<E> subList(int fromIndex,
                       int toIndex)
```

Returns a view of the portion of this list between the specified `fromIndex`, inclusive, and `toIndex`, exclusive. (If `fromIndex` and `toIndex` are equal, the returned list is empty.) The returned list is backed by this list, so non-structural changes in the returned list are reflected in this list, and vice-

versa. The returned list supports all of the optional list operations supported by this list.

This method eliminates the need for explicit range operations (of the sort that commonly exist for arrays). Any operation that expects a list can be used as a range operation by passing a subList view instead of a whole list. For example, the following idiom removes a range of elements from a list:

```
list.subList(from, to).clear();
```

Similar idioms may be constructed for `indexOf` and `lastIndexOf`, and all of the algorithms in the `Collections` class can be applied to a subList.

The semantics of the list returned by this method become undefined if the backing list (i.e., this list) is *structurally modified* in any way other than via the returned list. (Structural modifications are those that change the size of this list, or otherwise perturb it in such a fashion that iterations in progress may yield incorrect results.)

This implementation returns a list that subclasses `AbstractList`. The subclass stores, in private fields, the offset of the subList within the backing list, the size of the subList (which can change over its lifetime), and the expected `modCount` value of the backing list. There are two variants of the subclass, one of which implements `RandomAccess`. If this list implements `RandomAccess` the returned list will be an instance of the subclass that implements `RandomAccess`.

The subclass's `set(int, E)`, `get(int)`, `add(int, E)`, `remove(int)`, `addAll(int, Collection)` and `removeRange(int, int)` methods all delegate to the corresponding methods on the backing abstract list, after bounds-checking the index and adjusting for the offset. The `addAll(Collection c)` method merely returns `addAll(size, c)`.

The `listIterator(int)` method returns a "wrapper object" over a list iterator on the backing list, which is created with the corresponding method on the backing list. The `iterator` method merely returns `listIterator()`, and the `size` method merely returns the subclass's `size` field.

All methods first check to see if the actual `modCount` of the backing list is equal to its expected value, and throw a `ConcurrentModificationException` if it is not.

Specified by:

`subList` in interface `List<E>`

Parameters:

`fromIndex - low endpoint (inclusive) of the subList`

`toIndex - high endpoint (exclusive) of the subList`

Returns:

a view of the specified range within this list

Throws:

`IndexOutOfBoundsException` - if an endpoint index value is out of range (`fromIndex < 0 || toIndex > size`)

`IllegalArgumentException` - if the endpoint indices are out of order (`fromIndex > toIndex`)

equals

```
public boolean equals(Object o)
```

Compares the specified object with this list for equality. Returns `true` if and only if the specified object is also a list, both lists have the same size, and all corresponding pairs of elements in the two lists are *equal*. (Two elements `e1` and `e2` are *equal* if `(e1==null ? e2==null : e1.equals(e2))`.) In other words, two lists are defined to be equal if they contain the same elements in the same order.

This implementation first checks if the specified object is this list. If so, it returns `true`; if not, it checks if the specified object is a list. If not, it returns `false`; if so, it iterates over both lists, comparing corresponding pairs of elements. If any comparison returns `false`, this method returns `false`. If either iterator runs out of elements before the other it returns `false` (as the lists are of unequal length); otherwise it returns `true` when the iterations complete.

Specified by:

`equals` in interface `Collection<E>`

Specified by:

`equals` in interface `List<E>`

Overrides:

`equals` in class `Object`

Parameters:

`o` - the object to be compared for equality with this list

Returns:

`true` if the specified object is equal to this list

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns the hash code value for this list.

This implementation uses exactly the code that is used to define the list hash function in the documentation for the `List.hashCode()` method.

Specified by:

`hashCode` in interface `Collection<E>`

Specified by:

`hashCode` in interface `List<E>`

Overrides:

`hashCode` in class `Object`

Returns:

the hash code value for this list

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

removeRange

```
protected void removeRange(int fromIndex,
                           int toIndex)
```

Removes from this list all of the elements whose index is between `fromIndex`, inclusive, and `toIndex`, exclusive. Shifts any succeeding elements to the left (reduces their index). This call shortens the list by `(toIndex - fromIndex)` elements. (If `toIndex==fromIndex`, this operation has no effect.)

This method is called by the `clear` operation on this list and its subLists. Overriding this method to take advantage of the internals of the list implementation can *substantially* improve the performance of the `clear` operation on this list and its subLists.

This implementation gets a list iterator positioned before `fromIndex`, and repeatedly calls `ListIterator.next` followed by `ListIterator.remove` until the entire range has been removed. **Note: if `ListIterator.remove` requires linear time, this implementation requires quadratic time.**

Parameters:

`fromIndex - index of first element to be removed`

C

```
toIndex - index after last element to be removed
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.